

```
In [1]: from pathlib import Path
import sys, csv

THRESH = 3.5
ROOT = Path.cwd()
FREQ_DIR = ROOT / "freq-master"
CSV_PATH = ROOT / "top-1m.csv"
```

```
In [2]: sys.path.insert(0, str(FREQ_DIR))
from freq import FreqCounter

fc = FreqCounter()
fc.load(str(FREQ_DIR / "freqtable2018.freq"))

def freq_percent(s: str) -> float:
    # average bigram probability (%) for a string
    return float(fc.probability(s)[0])
```

```
In [3]: import tldextract, pandas as pd
ext = tldextract.TLDExtract(cache_dir=None, suffix_list_urls=None)

def label_strict(host: str) -> str:
    # only the domain label (e.g., www.google.com -> google), empty if not parseable
    return ext(str(host)).domain.lower()

PREVIEW_N = 5

rows, cache = [], {}
with CSV_PATH.open("r", encoding="utf-8", newline="") as f:
    for row in csv.reader(f):
        if len(row) < 2:
            continue
        url = row[1].strip()
        lbl = label_strict(url)
        if not lbl:
            continue
        if lbl not in cache:
            cache[lbl] = round(freq_percent(lbl), 4) # compute once per label
        rows.append({"URL": url, "Domain Name": lbl, "frequency": cache[lbl]})
        if len(rows) == PREVIEW_N:
            break

pd.DataFrame(rows)
```

Out[3]:

	URL	Domain Name	frequency
0	google.com	google	6.6142
1	microsoft.com	microsoft	6.1558
2	www.google.com	google	6.6142
3	facebook.com	facebook	6.4581
4	doubleclick.net	doubleclick	7.7576

In [4]:

```
labels = set()
with CSV_PATH.open("r", encoding="utf-8", newline="") as f:
    for row in csv.reader(f):
        if len(row) < 2:
            continue
        lbl = label_strict(row[1].strip())
        if lbl:
            labels.add(lbl)

count_below = sum(1 for lbl in labels if freq_percent(lbl) < THRESH)
print(count_below)
```

32898

In []: